Computer Networks Lab (CSE/PC/B/S/314): Assignment 1 Report

Error Detection Module: Checksum and Cyclic Redundancy Check (CRC)

1. Title

Field	Details
Name	SOUMYADIP MODAK
Roll	302510501001
Class	BCSE / 3year / Semester 1
Group	A1
Assignment number	1 (CO1)
Course	CSE/PC/B/S/314 Computer Networks Lab
Code demonstration deadline	10-14 August 2026
Report submission deadline	17-21 August 2026
Date of submission	
Language / platform	C, POSIX UDP sockets (Linux)

Problem statement. Design and implement an error detection module with two schemes: a 16-bit Internet-style Checksum and a Cyclic Redundancy Check (CRC-8, CRC-10, CRC-16, CRC-32). A Sender reads a test file, builds Ethernet-sized frames (minimum 46-byte payload), appends the error-detection codes, injects errors through a separate error-injection method and transmits the frames. A Receiver recomputes both codes and accepts or rejects each frame. The two schemes are then compared for single-bit errors, two isolated single-bit errors, an odd number of errors and burst errors, in terms of detection capability, execution time and resource usage.

2. Design

2.1 Purpose of the program

The program is a client-server simulation of an unreliable link. The **Sender (Client.c)** splits a file into 46-byte data words, wraps each in a fixed frame, computes a 16-bit checksum and a user-selected CRC, optionally corrupts the frame with one of four error models and sends it over UDP (port 4455). The **Receiver (Server.c)** recomputes the checksum and the CRC on every frame, rejects the frame if either check fails, and records which scheme(s) detected the error. At the end it prints a summary of how often each scheme caught the errors.

2.2 Frame structure

Each frame is built as: header + payload + checksum + CRC.

Field	Size (bytes)	Notes
Header	16	Zero-filled placeholder
Payload	46	Next 46 bytes of the file; last chunk zero-padded
Checksum	2	16-bit one's complement checksum over header + payload (62 bytes), little-endian
CRC	1 / 2 / 2 / 4	CRC-8 / CRC-10 / CRC-16 / CRC-32, little-endian
Total	65 / 66 / 66 / 68	Within the 64 to 1500 byte Ethernet range

The end of transmission is signaled by a 4-byte datagram containing the text `DONE`.

2.3 Structure diagram (procedural organization)

```
SENDER (Client.c)
main()
  |-- parse argv (receiver IP, test file), open file, create UDP socket
  |-- menu: CRC type (8/10/16/32), error mode (random / selected), error type
  `-- send_frames()
       |-- fread 46 bytes (zero-pad last chunk)
       |-- build frame: [16B header][46B payload]
       |-- cal_chksum()          -> append 2 bytes      (checksum.h)
       |-- cal_crc8/10/16/32()   -> append 1/2/4 bytes  (Crc.h)
       |-- injkterror()          -> corrupt frame by chosen error model
       `-- sendto() + usleep(50 ms)  ... then send "DONE"
```

```
RECEIVER (Server.c)
main()
  |-- menu: CRC type, create + bind UDP socket on port 4455
  `-- loop: recvfrom()
       |-- "DONE"              -> break, print final results
       |-- size != expected     -> report malformed frame
       |-- cal_chksum() over 64 bytes   -> chk_error  (result != 0)
       |-- cal_crc*()   over 62 bytes   -> crc_error  (calc != received)
       `-- classify: both / checksum-only / CRC-only / missed-both,
                then REJECT or ACCEPT frame
```

2.4 Input and output format

Sender input: command line `./client <Receiver_IP> <Test_File>`, then interactive choices: CRC width (8, 10, 16 or 32); error mode (1 = random type per frame, 2 = one selected type for all frames); error type (1 = none, 2 = single-bit, 3 = two isolated single-bit, 4 = odd number (3 bits), 5 = burst).

Sender output: one line per frame stating the injected error (type, byte and bit positions) and the number of bytes sent, followed by a completion message.

Receiver input: `./server`, then the CRC width, which must match the Sender.

Receiver output: one line per frame (`REJECTED` or `accepted`) and a final summary: total frames evaluated, and counts and percentages for *Both detected*, *Checksum only*, *CRC only* and *Missed by both*.

Test data: `Test.txt`, 30,720 bytes, giving 668 frames (667 full frames plus one zero-padded frame of 38 bytes). Results were collected in `Data.csv` (an Excel workbook).

3. Implementation

3.1 Checksum (checksum.h)

`cal_chksum(frame, length)` adds the data as 16-bit words into a 32-bit accumulator (an odd trailing byte is added on its own), folds the carries back in (`while (sum >> 16)`, which is one's complement addition) and returns the complement of the sum. The Sender appends the checksum to the frame. At the

Receiver the same function is run over the first 64 bytes (data + checksum field); an error-free frame yields exactly 0x0000, so any non-zero result means an error.

3.2 CRC (Crc.h)

All four CRCs are bitwise, MSB-first, shift-register implementations with initial value 0 and no final XOR. The generator polynomials follow the assignment:

CRC	Polynomial (as given)	Constant used	Register width
CRC-8	$x^8 + x^7 + x^6 + x^4 + x^2 + 1$	0xD5	8 bits
CRC-10	$x^{10} + x^9 + x^5 + x^4 + x + 1$	0x233	10 bits (masked with 0x3FF)
CRC-16	$x^{16} + x^{15} + x^2 + 1$	0x8005	16 bits
CRC-32	$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$	0x04C11DB7	32 bits

For each input byte the register is XOR-ed with the byte (aligned to the top of the register), then shifted 8 times; whenever the top bit is 1 the polynomial is XOR-ed in after the shift. The CRC is computed over the 62-byte header + payload.

3.3 Error injection (Inject error in Client.c)

A separate method injecterror(frame, length, choice) flips bits of the finished frame (including the checksum and CRC fields):

Choice	Error model	Method
1	No error	Frame unchanged
2	Single-bit	One random bit of one random byte is flipped
3	Two isolated single-bit	One bit flipped in the first half and one in the second half of the frame
4	Odd number of errors	Three random bit flips
5	Burst	XOR 0x1E on one random byte: 4 consecutive bits flipped

In random mode the Sender picks a random error type (1 to 5) for every frame.

3.4 Communication

Frames travel in UDP datagrams (`sendto` / `recvfrom`) with a 50 ms delay between frames. The Receiver identifies the termination datagram `DONE` and checks that every other datagram has the exact expected length for the chosen CRC; other sizes are reported as malformed.

3.5 Classification at the Receiver

For every frame the Receiver computes `chk_error` (checksum result is non-zero) and `crc_error` (recomputed CRC differs from the received CRC) and counts: **both** detected; **checksum only**; **CRC only**; and **neither** (error missed by both).

4. Test Cases

Every test uses `Test.txt` (668 frames) and is repeated for each CRC width (8, 10, 16, 32) by running the Receiver and the Sender with the same choice.

#	Test	Input / setting	What is checked
T1	No error	Error type 1	Checksum result is `0x0000`, CRC matches, all frames accepted (no false rejection)
T2	Single-bit error	Error type 2	One flipped bit is detected; the CRC should catch 100% of these
T3	Two isolated single-bit errors	Error type 3	Errors in different halves of the frame are detected
T4	Odd number of errors	Error type 4 (3 bits)	Odd weight errors are detected (a CRC with an (x+1) factor catches all)
T5	Burst error	Error type 5 (4-bit burst)	Burst shorter than the CRC width is always caught by the CRC
T6	Random mix	Mode 1 (random per frame)	All error types appear together and the counters add

#	Test	Input / setting	What is checked
			up to the total
T7	Last-chunk padding	File length not a multiple of 46	The final 38-byte chunk is zero-padded to 46 bytes and the frame is valid
T8	Wrong CRC selection	Sender and Receiver choose different widths	Frame length mismatch is reported as *malformed frame*
T9	Termination	End of file	DONE datagram stops the Receiver and the summary is printed
T10	Invalid menu input	CRC choice other than 8/10/16/32	Program falls back to CRC-16
T11	Error in the check fields	Error lands in checksum or CRC bytes	Shows which scheme covers which part of the frame (see Analysis)
T12	Timing	Isolated benchmark of each function	Execution time per 62-byte frame

5. Results

5.1 Performance metrics

- **Detection percentage** for each category, over 668 frames per configuration: *both detected*, *checksum only*, *CRC only*, *missed by both*.
- **Execution time** of each checksum / CRC function per 62-byte frame.
- **Resource usage:** memory and code footprint.

5.2 Detection results (from Data.csv, % of 668 frames)

Single-bit error

CRC	Both detected	Checksum only	CRC only	Missed by both	Total frames
CRC-8	95.66	3.14	1.20	0	668

CRC	Both detected	Checksum only	CRC only	Missed by both	Total frames
CRC-10	96.26	0.90	2.84	0	668
CRC-16	97.90	0	2.10	0	668
CRC-32	95.21	0	4.79	0	668

Two isolated single-bit errors

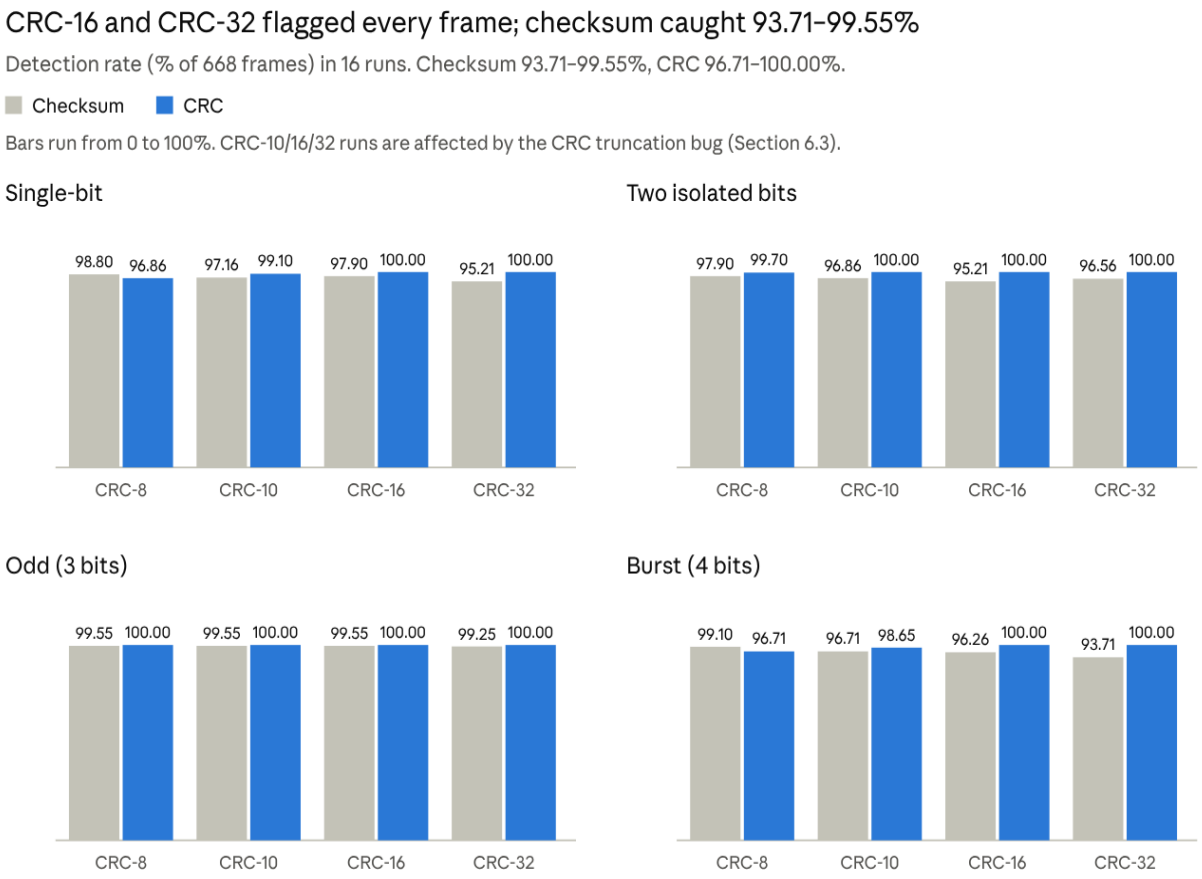
CRC	Both detected	Checksum only	CRC only	Missed by both	Total frames
CRC-8	97.60	0.30	2.10	0	668
CRC-10	96.86	0	3.14	0	668
CRC-16	95.21	0	4.79	0	668
CRC-32	96.56	0	3.44	0	668

Odd number of errors (3 bits)

CRC	Both detected	Checksum only	CRC only	Missed by both	Total frames
CRC-8	99.55	0	0.45	0	668
CRC-10	99.55	0	0.45	0	668
CRC-16	99.55	0	0.45	0	668
CRC-32	99.25	0	0.75	0	668

Burst error (4 bits)

CRC	Both detected	Checksum only	CRC only	Missed by both	Total frames
CRC-8	95.81	3.29	0.90	0	668
CRC-10	95.36	1.35	3.29	0	668
CRC-16	96.26	0	3.74	0	668
CRC-32	93.71	0	6.29	0	668



Data.csv, Sheet1 · 668 frames per run · checksum = both + checksum only, CRC = both + CRC only

The checksum's misses are mostly errors that land in the CRC bytes, which it does not cover; the CRC's misses at CRC-8 and CRC-10 are errors in the checksum bytes (Section 6.2).

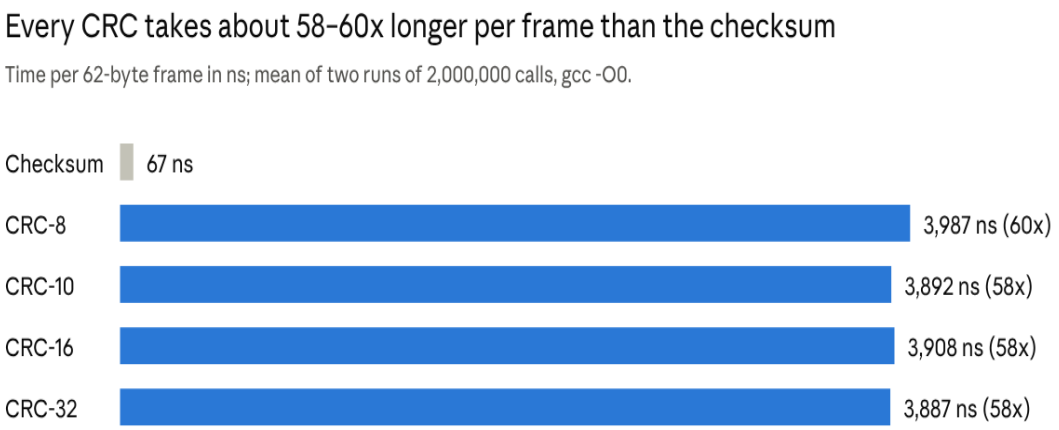
5.3 Execution time

Each function was timed in a separate benchmark (2,000,000 calls on a 62-byte buffer, compiled with `gcc -O0`, run twice on the same machine). Re-run it on your lab machine and replace these numbers if they differ.

Scheme	Time per frame (ns)	Relative to checksum
Checksum (16-bit)	about 67	1x
CRC-8	about 3,940 to 4,040	about 59x to 60x

Scheme	Time per frame (ns)	Relative to checksum
CRC-10	about 3,850 to 3,930	about 57x to 59x
CRC-16	about 3,890 to 3,930	about 58x to 59x
CRC-32	about 3,870 to 3,900	about 58x

All four CRCs cost about the same, because the bitwise loop does 8 shift-and-test steps per byte whatever the register width. The checksum is far cheaper since it does one 16-bit addition per two bytes.



benchmark of checksum.h and Crc.h, 62-byte frame · re-run on your lab machine to replace

5.4 Resource utilisation

Resource	Checksum	CRC (all widths)
Working memory	one 32-bit accumulator	one 8 to 32-bit register
Lookup tables	none	none (bitwise algorithm)
Per-frame buffer	70 bytes in the Sender, 100 bytes in the Receiver	same
Overhead added to frame	2 bytes	1, 2, 2 or 4 bytes

Both schemes use constant extra memory, so memory is not a differentiator. The difference is CPU time (Section 5.3). A table-driven CRC (256-entry table) would

trade 256 to 1024 bytes of memory for roughly an 8-fold reduction in loop iterations.

6. Analysis

6.1 Reading the results

The *missed by both* column is 0 in every test, and for every error type 100% of frames were rejected by at least one scheme. In theory a CRC with a suitable generator detects every single-bit error, every burst no longer than its width, and (when divisible by x+1) every odd-weight error; the Internet checksum detects every single-bit error and all bursts shorter than 16 bits but is weaker against compensating changes. With only 668 frames and random positions, the differences in the percentages between CRC widths in Section 5.2 are mostly sampling noise, not properties of the polynomials.

Theoretical guarantee versus observed CRC detection (% of 668 frames)

Error type	Theoretical CRC guarantee for this frame size	CRC-8	CRC-10	CRC-16	CRC-32
Single-bit	100% for every CRC (the generator has more than one term)	96.86	99.10	100.00	100.00
Two isolated bits	100% when the polynomial's period exceeds the covered length (about 500 bits): CRC-8 period 93 (no guarantee), CRC-10 511, CRC-16 32,767, CRC-32 far larger	99.70	100.00	100.00	100.00
Odd (3 bits)	CRC-8, CRC-10, CRC-16: 100%, because (x+1) divides the generator. CRC-32 has 15 terms, so (x+1) does not divide it; it still catches every 1 to 3-bit error at this length (Hamming distance 4 or	100.00	100.00	100.00	100.00

Error type	Theoretical CRC guarantee for this frame size	CRC-8	CRC-10	CRC-16	CRC-32
	more)				
Burst (4 bits)	100% for every CRC (burst length 4 is not more than the CRC width)	96.71	98.65	100.00	100.00

Observed CRC detection is the *both detected* share plus the *CRC only* share from Section 5.2. The CRC-8 and CRC-10 shortfalls below 100% come from errors that land in the checksum bytes, which the CRC does not cover (Sections 6.2 and 6.3), not from the polynomials.

6.2 Why "checksum only" and "CRC only" are not zero

The two schemes protect different byte ranges, and the error injector corrupts the whole frame:

- The CRC is computed over bytes 0 to 61, so it does **not** cover the 2 checksum bytes. An error that lands in the checksum field is caught by the checksum but not by the CRC, which gives the *checksum only* counts. For CRC-8 this is about 3.1% for single-bit and burst errors, which matches 2 of 65 bytes (3.1%).
- The checksum is checked over bytes 0 to 63, so it does **not** cover the CRC field. An error that lands in the CRC bytes is caught by the CRC but not by the checksum, which gives the *CRC only* counts (about 1.5%, 2.9% and 5.9% of bytes for CRC-8, 16 and 32 respectively).

So "detected by one scheme but not the other" in this experiment mostly reflects *where* the error landed, and not a weakness of either algorithm.

6.3 Known bug that affects the CRC-10, CRC-16 and CRC-32 results

In `send_frames()` the variable holding the CRC is declared as `unsigned char crc;` and is then assigned the return value of `cal_crc10`, `cal_crc16` and `cal_crc32`. This truncates the CRC to its lowest byte, so the upper bytes written to the frame (`crc >> 8`, `crc >> 16`, `crc >> 24`) are always 0. The Receiver compares against the full-width CRC, so for these three widths the CRC check will fail on almost every

frame, including error-free ones (for example the CRC-16 of a test frame is `0x9D6F` but only `0x6F` is stored).

Evidence in the results: the *checksum only* column is non-zero only for CRC-8 (3.14%, single bit). For CRC-10 it is 0.90%, which is about a quarter of the 3.1% seen for CRC-8, matching the 1-in-4 chance that the two missing high bits of a 10-bit CRC happen to be zero. For CRC-16 and CRC-32 it is exactly 0. A correct CRC would leave the same *checksum only* share as CRC-8 (errors landing in the checksum bytes). Hence the high detection rates of CRC-10, CRC-16 and CRC-32 are partly an artefact of this bug, and Test T1 (no error) is expected to fail for these widths.

Fix: declare `uint32_t crc;` in `send_frames()`. After the fix the experiments must be re-run before the numbers in Section 5.2 are final.

6.4 Other known issues

- In `Server.c`, the *missed by both* counter sits in an `else` branch that can never execute (it is inside `if (chk_error || crc_error)`), so it always prints 0 regardless of reality. Counting frames that are known to be corrupted but accepted needs the Sender's injection information.
- The final-results `printf` lines for *Checksum only* and *CRC only* post-increment the counters inside the call, which is undefined behaviour and may print inconsistent values.
- The two-error injector places one error in each half of the frame and the three-error injector may pick the same bit twice, which cancels out, so "3 errors" can occasionally be 1 error.
- The burst injector flips a fixed 4-bit pattern inside one byte, so longer bursts (such as 16 or 32 bits, which are where CRC-16 and CRC-32 differ from CRC-8) are never tested.
- Checksum and CRC polynomials are hard-coded rather than read as input, and UDP loss or reordering is not handled.

6.5 Execution time and trade-offs

The checksum is about 58 times faster than any of the bitwise CRCs on a 62-byte frame, at the price of a weaker guarantee. CRC-32 gives the strongest protection

for the same compute cost as CRC-8 here but needs 4 bytes of overhead per frame. For small frames, CRC-16 is a good compromise.

6.6 Possible improvements

1. Fix the `unsigned char crc` bug and re-run all experiments with many more frames (for example 10^5 or more per case).
2. Make the CRC cover the whole frame including the checksum, or inject errors only into the header and payload, so the two schemes are compared on identical bytes.
3. Have the Sender log what it injected so the Receiver can compute a true *missed by both* count.
4. Add burst errors of length 8, 16, 17 and 33 bits to test the theoretical burst-length limits of each CRC.
5. Use a table-driven CRC for speed and read the polynomials from input as the assignment states.

7. Comments

This assignment showed how the same frame can be protected by two different codes and how the choice of code and of the bytes it covers changes what is detected. Implementing the CRC as a shift register made the polynomial division concrete, and debugging the results showed how easily a small type mistake (an 8-bit variable holding a wider CRC) silently changes an experiment. The assignment was of moderate difficulty: the algorithms are short, but designing a fair experiment (what each code covers, how errors are classified) is the harder part. A suggestion is to specify in advance which bytes each scheme must protect, and to ask for error lengths that actually separate CRC-8, CRC-16 and CRC-32.

Appendix: source files

`Client.c` (Sender and error injection), `Server.c` (Receiver and classification), `checksum.h` (16-bit checksum), `Crc.h` (CRC-8/10/16/32), `Test.txt` (test input, 30,720 bytes), `Data.csv` (results workbook).